

Hindsight — Architecture & Security Audit

Full-spectrum engineering assessment: security, architecture, code quality, scaling, maintainability, dependency health, and test coverage across the Hindsight codebase.

AUDIT SCOPE

Full-spectrum audit of the Hindsight codebase: security, architecture, code quality, scaling, maintainability, dependency health, and test coverage. Production code covering hindsight-api, hindsight-integrations, hindsight-clients, and supporting modules. Four-tier adversarial review (P0 → P1 → P2 → Pass spot-check) with deterministic scoring.

● Overview

● Security (9)

● Architecture (7)

● Code Quality (3)

● Scaling (4)

● Maintainability (3)

● Technical Debt (3)

● Dependencies (3)

● Testing (4)

● Founder Report →

01

Technical Report Summary

Audit date: 2026-05-27. Four-tier adversarial review completed: P0 challenge, P1 challenge, P2 challenge, and Pass spot-check. Final distribution: **0 P0, 5 P1, 21 P2, 3 P3, 7 PASS**. Overall health: **84.9/100 (Grade B)**.

0

CRITICAL (P0)

5

WARNING (P1)

21

PLAN (P2)

7

PASSED

CATEGORY	SCORE	GRADE	P0	P1	P2	P3	FINDINGS
Security	75	C+	0	1	6	1	9
Architecture	81	B-	0	1	4	0	7
Code Quality	94	A	0	0	2	0	3
Scaling	84	B	0	1	3	0	4
Maintainability	97	A+	0	0	1	1	3
Technical Debt	91	A-	0	0	3	0	3
Dependency Health	90	A-	0	1	1	0	3
Testing	90	A-	0	1	1	1	4

SECURITY — 75/100

02 Security Findings

9 findings: 1 P1 (no CORS or rate limiting), 6 P2, 1 P3, 1 PASS. The default deployment is open to cross-origin requests and has no per-second rate limiting. Individual findings are moderate in isolation but compound when combined (SEC-01 + SEC-03 + SEC-09 = unauthenticated access with schema leakage).

P1 SEC-03 — No CORS configuration or rate limiting on API endpoints → [Founder view](#)

confirmed P1

File: hindsight-api/hindsight_api/api/http.py:1797

Technical description: The FastAPI create_app function does not add CORSMiddleware or any rate limiting middleware. grep for CORS, CORSMiddleware, rate_limit, and slowapi across the entire hindsight_api/ directory returns zero results. The only middleware added is http_metrics_middleware (Prometheus telemetry).

Compound risk: Combined with SEC-01 (default no-auth) and SEC-09 (raw exception messages), any internet-reachable deployment is fully exposed — any origin can call the

API, with unlimited request frequency, and receive database schema details in error responses.

Internal backpressure exists but is insufficient: Internal semaphores (`_global_llm_semaphore = 32`, `_search_semaphore = 32`, `_put_semaphore = 5`) provide process-level concurrency limits but are NOT per-tenant and NOT per-second rate limits. A single tenant can consume all semaphore slots.

```
hindsight-api/hindsight_api/api/http.py:1797
1797 app = FastAPI(title=..., lifespan=lifespan)
1798 app.add_middleware(http_metrics_middleware) # Only middleware – no
CORs, no rate limiting
```

Recommended fix: Add `CORSMiddleware` with explicit `allowed_origins`. Add rate limiting via `slowapi`. Make both configurable via environment variables.

P2 SEC-01 – `DefaultTenantExtension` allows unauthenticated access to all data

→ [Founder view](#) downgraded from P1

File: `hindsight-api/hindsight_api/extensions/builtin/tenant.py:26`

Technical description: `DefaultTenantExtension.authenticate()` returns `TenantContext(schema_name=self._schema)` without any authentication check. When no tenant extension is configured, all requests are served on the default schema with no API key or token validation.

Review note: Downgraded from P1 because `DefaultTenantExtension` is documented as no-auth single-tenant for development. The real risk is deployment misconfiguration. A startup WARNING log when `DefaultTenantExtension` is active would be the appropriate remediation.

```
hindsight-api/hindsight_api/extensions/builtin/tenant.py:26
26 async def authenticate(self, request: Request) -> TenantContext:
27     return TenantContext(schema_name=self._schema)
```

Recommended fix: Add a startup WARNING log when `DefaultTenantExtension` is active. Consider requiring explicit opt-in for no-auth mode in production.

P2 SEC-02 – MCP auth can be disabled via environment variable → [Founder view](#)

downgraded from P1

File: `hindsight-api/hindsight_api/extensions/builtin/tenant.py:67`

Technical description: `ApiKeyTenantExtension` has `mcp_auth_disabled` flag. When set to `true/1/yes`, `authenticate_mcp` returns `TenantContext` without checking the API key. Documented as backward compatibility for MCP transports that provide their own auth layer.

Review note: Downgraded from P1 — explicit opt-in flag (default empty/false). Only needed for stdin-based MCP servers. A startup WARNING when enabled would be appropriate.

Recommended fix: Add a startup WARNING when `mcp_auth_disabled` is set. Consider deprecating this flag.

P2 SEC-04 – SQL table names constructed via f-string interpolation → [Founder view](#)

File: `hindsight-api/hindsight_api/engine/memory_engine.py:59` (and 4 other locations)

Technical description: `fq_table(table_name)` returns `f'{schema}.{table_name}'` where `table_name` comes from hardcoded `_PROTECTED_TABLES` whitelist. The pattern is used in 5 modules (131 call sites total). Current implementation is safe because table names come from whitelist, but f-string interpolation for SQL identifiers is a category error that creates a maintenance hazard.

Recommended fix: Migrate `fq_table()` to use `asyncpg's quote_ident` or SQLAlchemy's `quote`. Document that `fq_table()` must only be called with hardcoded table names.

P2 SEC-05 – Admin CLI TRUNCATE uses f-string interpolation for schema/table names

→ [Founder view](#)

File: `hindsight-api/hindsight_api/admin/cli.py:115`

Technical description: Admin CLI uses f-string interpolation for `TRUNCATE TABLE {qualified_table} CASCADE` and `REFRESH MATERIALIZED VIEW`. Attack surface is minimal (local admin tool only, schema from config/env, table names from hardcoded lists), but TRUNCATE is destructive and proper identifier quoting is best practice.

Recommended fix: Use `asyncpg's quote_ident` or validate schema/table names against an allowlist before interpolation.

P2 SEC-07 – Default PostgreSQL password in Helm values → [Founder view](#)

File: `helm/hindsight/values.yaml:213`

Technical description: password: 'hindsight' is the default PostgreSQL password in the Helm chart. Many Kubernetes deployments ship with default values. PostgreSQL is network-accessible within the cluster.

Recommended fix: Remove the default password. Make it required, or generate a random one on install. Add a NOTES.txt warning.

P2 SEC-09 – HTTP 500 responses expose raw exception messages to API clients

→ **Founder view** **found by spot-check**

File: hindsight-api/hindsight_api/api/http.py:1987 (45 instances)

Technical description: http.py contains 45 instances of `HTTPException(status_code=500, detail=str(e))` that expose raw Python exception messages to API clients. Database errors (`asyncpg.PostgresError`) can include table names, column names, constraint names, and schema details. Compounds with SEC-01 (default no-auth) and SEC-03 (no CORS/rate-limiting).

```
hindsight-api/hindsight_api/api/http.py:1987
1987 except Exception as e:
1988     import traceback
1989     error_detail = str(e) + traceback.format_exc()
1990     logger.error(...)
1991     raise HTTPException(status_code=500, detail=str(e))
```

Recommended fix: Add a global exception handler to FastAPI that sanitizes 500 error responses. Replace `detail=str(e)` with a generic message in production. Log full exceptions server-side (already done).

P3 SEC-06 – dotenv override=True may leak env vars in multi-tenant deployments

downgraded from P2

File: hindsight-api/hindsight_api/config.py:17

Technical description: `load_dotenv(find_dotenv(usecwd=True), override=True)` allows `.env` values to take precedence over environment variables. Hindsight is single-tenant per process — the multi-tenant framing is incorrect. The pattern is typically desired for local development. In containerized deployments, `.env` files are not typically mounted.

Review note: Downgraded to P3 — the risk requires local filesystem access and overrides the typical desired direction.

PASS**SEC-08 – Security credentials properly managed via environment variables**

All API keys, database URLs, and service account keys are loaded from environment variables with no hardcoded defaults. `_CREDENTIAL_FIELDS` clearly marks fields that must never be exposed via API. `.env` file is in `.gitignore`.

ARCHITECTURE – 81/100

03

Architecture Findings

7 findings: 1 P1 (7000-line god class), 4 P2, 2 PASS. The `MemoryEngine` god class is the dominant structural risk, affecting development velocity, test coverage, and regression risk.

P1**ARCH-01 – `MemoryEngine` is a 7000-line god class with 97 methods and 112 imports**[→ Founder view](#)

File: `hindsight-api/hindsight_api/engine/memory_engine.py:223`

Technical description: `MemoryEngine` is 6,979 lines with 98 method definitions and 112 import lines. It handles connection pool management, async operation tracking, retain/recall/reflect orchestration, consolidation, bank CRUD, document management, entity resolution, and health checks in a single class. The 21 bare except `Exception` blocks (see CQ-01) make it difficult to reason about error propagation.

```
hindsight-api/hindsight_api/engine/memory_engine.py
223 class MemoryEngine(MemoryEngineInterface):
...   # 6979 lines, 98 methods, 112 imports, 21 bare except blocks
```

Recommended fix: Decompose following Interface Segregation Principle. Extract into focused modules: `RetainEngine`, `RecallEngine`, `ReflectEngine`, `BankManager`, `DocumentManager`, `OperationManager`, `ConnectionManager`. The existing `MemoryEngineInterface` ABC provides a good starting contract. See TD-01 for the decomposition plan.

P2**ARCH-02 – Direct `asyncpg` SQL in business logic bypasses ORM abstraction**[→ Founder view](#)[downgraded from P1](#)

File: `hindsight-api/hindsight_api/engine/memory_engine.py`

Technical description: `MemoryEngine` uses exclusively `asyncpg` for runtime data access. SQLAlchemy models in `models.py` (316 lines) are used only for Alembic

migrations, not runtime queries — so there is no true "dual pattern." However, schema changes must be coordinated between Alembic migrations and 99+ asyncpg SQL strings, creating a maintenance risk. The `fq_table()` pattern with `_PROTECTED_TABLES` whitelist provides defense in depth.

Review note: Downgraded from P1 because there is not a dual runtime data access pattern — SQLAlchemy models are schema-only. The concern about schema coordination is legitimate but subsumed by ARCH-01's decomposition.

Recommended fix: Create a Repository layer that encapsulates all asyncpg SQL, with MemoryEngine calling repository methods instead of raw SQL strings. This would naturally emerge from ARCH-01 decomposition.

P2

ARCH-03 – `http.py` is a 4200-line monolithic API module combining routes, models, and business logic

→ [Founder view](#)

File: `hindsight-api/hindsight_api/api/http.py` (4,230 lines, 138 function definitions)

Technical description: `http.py` combines FastAPI route definitions, Pydantic request/response models, authentication middleware, and business logic in a single file. The file is well-structured internally with clear sections, but navigation and isolated testing are difficult.

Recommended fix: Split into module-based routers following FastAPI's `APIRouter` pattern: `routers/retain.py`, `routers/recall.py`, `routers/reflect.py`, `routers/banks.py`, `routers/documents.py`, `routers/admin.py`. Keep models in a separate `models/` module.

P2

ARCH-04 – `config.py` is 1300 lines with 100+ fields and manual env parsing

→ [Founder view](#)

File: `hindsight-api/hindsight_api/config.py` (1,308 lines, 178 `os.getenv/os.environ` calls)

Technical description: The `HindsightConfig` dataclass contains 100+ fields with manual `os.getenv()` calls in `from_env()`. Each new configuration option requires changes in 3 places: `ENV_` constant, `DEFAULT_` constant, and `from_env()` assignment. `ConfigFieldAccessProxy` is a well-designed access control layer that prevents misconfiguration.

Recommended fix: Refactor into configuration groups (e.g., `LLMConfig`, `EmbeddingsConfig`, `RetainConfig`) using `pydantic-settings` or similar typed

configuration library.

P2 ARCH-05 – `fq_table()` utility duplicated across 5 modules without shared module

→ [Founder view](#)

Files: `memory_engine.py:59`, `poller.py:28`, `task_backend.py:21`,
`storage/postgresql.py:15`, `admin/cli.py:21` (131 total call sites)

Technical description: `fq_table()` is implemented independently in 5 locations with different signatures: `memory_engine.py` uses context variables for schema, others take schema as a parameter. The duplication creates inconsistency risk.

Recommended fix: Extract `fq_table()` into a shared `db_utils.py` module. Standardize on the context-variable version as canonical.

PASS ARCH-06 – Well-structured bounded contexts for integrations and clients

Integration packages (`crewai`, `litellm`, `pydantic-ai`, `ai-sdk`, `chat`, `openclaw`) are cleanly separated with their own `pyproject.toml`, dedicated test suites, and minimal cross-dependencies. Client SDKs (Python, TypeScript, Rust, Go) are auto-generated from OpenAPI specs.

PASS ARCH-07 – Good use of ABC interface for `MemoryEngine` public API

`MemoryEngineInterface` (585 lines) properly defines the public contract with abstract methods, type hints, and comprehensive docstrings. This separates interface from implementation and enables mock testing — though the mock seam is currently unused (see TST-01).

CODE QUALITY – 94/100

04 Code Quality Findings

3 findings: 2 P2, 1 PASS. Code quality is strong overall. The two P2 findings are concentrated in the `MemoryEngine` god class and will be addressed by its decomposition (ARCH-01).

P2 CQ-01 – `MemoryEngine` has 21 bare `except Exception` blocks that swallow errors

downgraded from P1

File: hindsight-api/hindsight_api/engine/memory_engine.py (lines 657, 750, 919, 956, 981, 1007, 1039, 1066, 1172, 1450, 1791, 1800, 2071, 2112, 2140, 2167, 2955, 3270, 4792, 5166, and 1 more)

Technical description: 21 instances of `except Exception as e:` in `memory_engine.py`. While each logs the error, the pattern makes it difficult to distinguish expected failures from unexpected bugs. Some catch blocks retry the operation silently; others log and return partial results.

Review note: Downgraded from P1 — most are deliberate firewalls (extension error handlers, operation cleanup, health checks). They log the error rather than silently swallowing. Address during ARCH-01 decomposition by catching more specific exception types.

Recommended fix: Catch specific exception types (`asyncpg.PostgresError`, `ConnectionError`, custom `LLMError`). Create a custom exception hierarchy.

P2 CQ-02 – Broad exception handling in LLM providers

File: hindsight-api/hindsight_api/engine/providers/openai_compatible_llm.py

Technical description: `openai_compatible_llm.py` catches generic `Exception` in 4 places (lines 138, 490, 631, 782) while `anthropic_llm.py` catches specific exceptions in all paths. The provider also catches specific exceptions (`JSONDecodeError`, `LengthFinishReasonError`, `APIConnectionError`, `APIStatusError`, `httpx.HTTPStatusError`), but falls back to broad `Exception`.

Recommended fix: Standardize exception handling across all LLM providers. Create a common `LLMProviderError` base class with specific subtypes.

PASS CQ-04 – Good use of Pydantic models for API validation

HTTP endpoints use Pydantic `BaseModel`, `Field`, and `field_validator` for request/response validation. The API contract is well-defined with type hints and validators.

05 Scaling Findings

4 findings: 1 P1 (no rate limiting), 3 P2. The internal semaphores provide process-level backpressure but lack per-tenant fairness and per-second rate limiting.

P1

SCA-01 – No rate limiting on LLM API calls – burst traffic can exhaust provider quotas

→ [Founder view](#)

File: `hindsight-api/hindsight_api/engine/memory_engine.py`

Technical description: No rate limiting middleware exists at the FastAPI layer. Internal semaphores (`_global_llm_semaphore=32`, `_search_semaphore=32`, `_put_semaphore=5`) provide process-level concurrency limits, but any single tenant can consume ALL semaphore slots. No per-second/per-minute/per-hour rate limiting exists. No tenant-level fairness in semaphore scheduling.

Compound risk: Combined with SEC-03 (no CORS + no rate limiting at HTTP layer) and SEC-01 (default no-auth), cost and availability are unguarded for any internet-reachable deployment.

Recommended fix: Add request rate limiting at the FastAPI middleware layer (e.g., `slowapi`). Implement fair scheduling in the worker to prevent one tenant from monopolizing slots. Consider per-tenant quotas for LLM calls.

P2

SCA-02 – Connection pool max size (100) may be insufficient under high concurrency

File: `hindsight-api/hindsight_api/config.py` (`DEFAULT_DB_POOL_MAX_SIZE=100`)

Technical description: With `recall_max_concurrent=32`, worker connections, and health checks, 100 concurrent connections is reasonable for PostgreSQL. However, no circuit breaker pattern exists for database connection failures. Under load, degraded connections could exhaust the pool.

Recommended fix: Add connection pool metrics/logging. Consider implementing a circuit breaker. Make pool size more configurable based on deployment scale.

P2

SCA-03 – Embedding and reranking model loads are synchronous in `MemoryEngine.initialize()`

File: `hindsight-api/hindsight_api/engine/memory_engine.py:1203`

Technical description: `init_embeddings()` and `init_cross_encoder()` load models during `initialize()`. The `lazy_reranker` flag can defer `cross_encoder` init, but embeddings are always synchronous (~5-15 seconds for local models).

Recommended fix: Add lazy initialization for the embedding model as well. External embedding providers skip local loading entirely, so this only affects local model deployments.

P2 SCA-04 – No request body size limiting middleware on API endpoints

found by spot-check

File: hindsight-api/hindsight_api/api/http.py:1797

Technical description: No global request body size limiting middleware exists. File uploads have per-batch size limits (`file_conversion_max_batch_size_mb=50MB`), but general JSON request bodies (`retain/recall/reflect`) have no size limit. Combined with SEC-03 (no rate limiting), an unauthenticated client can send unlimited large payloads.

Recommended fix: Add request body size limiting middleware. Configure limits via environment variables (e.g., 10MB for JSON, 100MB for file uploads).

MAINTAINABILITY – 97/100

06

Maintainability Findings

3 findings: 1 P2, 1 P3, 1 PASS. Maintainability is strong — documentation and docstrings are comprehensive.

P2

MAI-01 – Monorepo with 9 Python packages and 3 JS/TS packages – no shared linting config in CI

Technical description: The repo contains `hindsight-api`, `hindsight`, `hindsight-embed`, `hindsight-cli`, `hindsight-clients`, `hindsight-integrations` (4 packages), `hindsight-dev`, and `hindsight-docs`. Each has its own `pyproject.toml`. A lint script exists at `scripts/hooks/lint.sh` and runs `ruff`, but it is only called in the `verify-generated-files` CI job, not as a general lint step.

Recommended fix: Add a root-level `ruff` configuration and run it in CI as a mandatory check.

P3 MAI-02 – Alembic migration files use non-sequential revision hashes

downgraded from P2

Technical description: Migration revision IDs are hex strings (e.g., 5a366d414dce) with no ordering convention. This is standard Alembic behavior — history is tracked via revision and down_revision fields. alembic history provides full navigation.

Review note: Downgraded to P3 — purely a DX preference for filename readability. Alembic tooling handles history traversal natively.

PASS MAI-03 – Good documentation and docstring coverage

Configuration has comprehensive docstrings. MemoryEngineInterface has thorough method-level documentation. Code is generally well-commented with clear intent. SECURITY.md provides a vulnerability reporting policy.

TECHNICAL DEBT – 91/100

07 Technical Debt Findings

3 findings: all P2. TD-01 overlaps with ARCH-01 (same root cause: the god class), but the decomposition plan perspective adds value.

P2 TD-01 – memory_engine.py is 7000 lines – decomposition deferred indefinitely

→ see ARCH-01 downgraded from P1

File: hindsight-api/hindsight_api/engine/memory_engine.py:223

Technical description: This finding tracks the *technical debt angle* (decomposition plan) of the god class issue covered by ARCH-01. While ARCH-01 addresses the structural risk, TD-01 tracks the deliberate deferral of decomposition and provides the phased plan.

Decomposition plan: (1) Extract WorkerManager into worker/task_executor.py, (2) Extract DocumentManager, BankManager into their own modules, (3) Extract ConnectionPool into db/connection.py, (4) Extract consolidation into consolidation/engine.py. MemoryEngine becomes a facade delegating to these modules.

P2 TD-02 – Cobuilder engine (25,000+ lines) lacks test coverage

File: cobuilder/engine/ (5,171 lines across pipeline_runner.py, session_runner.py, guardian.py)

Technical description: The `cobuilder/engine/` directory has 5,171 lines across core modules. Only 4 test files exist in `cobuilder/engine/conditions/tests/` (testing only the conditions subsystem). The orchestration, repomap, and pipeline subsystems have zero test coverage.

Recommended fix: Prioritize characterization tests for core paths: pipeline parsing, node/edge operations, signal protocol, and state transitions.

P2 TD-03 – OpenAPI client code is auto-generated but checked into version control

File: `hindsight-clients/python/hindsight_client_api/`

Technical description: Python and TypeScript clients in `hindsight-clients/` are auto-generated from OpenAPI specs. CI verifies generated code matches what `generate-clients.sh` produces, but the checked-in generated code creates git diff noise and a DX burden.

Recommended fix: Add generated client code to `.gitignore` and generate during build/publish. Alternatively, keep in git but add a strict CI check that regenerates and compares on every PR.

DEPENDENCY HEALTH – 90/100

08

Dependency Health Findings

3 findings: 1 P1 (torch as required dep), 1 P2, 1 PASS. The P1 is a deployment efficiency issue that affects all production images.

P1 DEP-02 – sentence-transformers and torch included as required dependencies

→ [Founder view](#) upgraded from P2

File: `hindsight-api/pyproject.toml` (lines 49-51)

Technical description: `sentence-transformers>=3.3.0` and `torch>=2.6.0` are listed as *required* dependencies. This means every production deployment — including those using external embedding providers (OpenAI, Anthropic) — must install a ~2GB dependency that is never used at runtime. The Docker `INCLUDE_LOCAL_MODELS=false` build arg can exclude model files, but the Python package still installs torch regardless.

Impact: (1) Docker image size inflated by ~2GB for ALL deployments, (2) torch has significant security surface area, (3) combined with SEC-03 and SCA-01, deployments that never use local models pay both cost and security overhead.

Recommended fix: Move sentence-transformers and torch to an extras_require group: `pip install hindsight-api[local-models]`. Make local embedding models optional rather than default.

P2 DEP-01 – Transitive dependency pins in pyproject.toml for CVE mitigations

File: hindsight-api/pyproject.toml

Technical description: Dependencies like pyasn1>=0.6.2, urllib3>=2.6.3, langchain-core>=1.2.5, filelock>=3.20.1, authlib>=1.6.6, aiohttp>=3.13.3 are pinned with CVE fix comments. These are transitive dependencies listed as direct dependencies to force specific versions.

Recommended fix: Use pip-audit or safety in CI for vulnerability detection. Consider a requirements-constraints file for version overrides instead of adding transitive deps to direct dependencies.

PASS DEP-03 – CI pipeline includes comprehensive multi-version Python testing

CI tests across Python 3.11, 3.12, 3.13, 3.14. Builds Docker images, runs integration tests, lint checks, and cross-client tests (Python, TypeScript, Rust, Go). Dependency security patches are explicitly tracked in pyproject.toml.

TESTING – 90/100

09

Testing Findings

4 findings: 1 P1 (no unit tests for god class), 1 P2, 1 P3, 1 PASS. The god class (ARCH-01) has zero dedicated unit tests, compounding its regression risk.

P1 TST-01 – Core engine (memory_engine.py, 7000 lines) has limited unit test coverage

→ [Founder view](#)

File: hindsight-api/tests/ (81 test files, all integration tests requiring PostgreSQL)

Technical description: 81 test files exist but are predominantly integration tests requiring a running PostgreSQL database (verified via EmbeddedPostgres fixture in conftest.py). Only 10 test files use Mock or patch. The 6,979-line MemoryEngine with 97 methods has zero dedicated unit tests. MemoryEngineInterface ABC (585 lines) exists as a clean mock injection seam but is completely unused for testing.

Compound risk: This directly enables ARCH-01's regression risk — without unit tests as a safety net, decomposition of the god class is dangerous.

Recommended fix: Add a unit test suite for MemoryEngine using mocked database connections. Test retain logic, recall logic, reflect orchestration, and consolidation independently. Use the Repository pattern to enable mock injection.

P2 TST-02 – No test coverage metrics or coverage enforcement in CI

File: `.github/workflows/test.yml`

Technical description: CI runs pytest but with no `--cov` flag, no coverage threshold, and no coverage reporting. Critical modules like `memory_engine.py` have zero dedicated unit test coverage.

Recommended fix: Add `pytest-cov` to test dependencies. Add coverage reporting to CI. Set minimum coverage thresholds for critical paths (e.g., `memory_engine.py`, `config.py`).

P3 TST-03 – `config_resolver` and hierarchical config lack dedicated test file

downgraded from P2

Technical description: `config_resolver.py` (275 lines) implements per-bank configuration override system. Tests exist in `test_hierarchical_config.py` and `test_config_validation.py`, but the test file name doesn't match the source file, reducing discoverability.

Review note: Downgraded to P3 — coverage exists, just not in a file named after `config_resolver.py`.

PASS TST-04 – Comprehensive CI testing across all client libraries and integrations

CI tests cover: API (Python 3.11-3.14), Python client, TypeScript client, Rust client, Go client, CLI smoke tests, CrewAI/LiteLLM/PydanticAI integrations, embed library, docs examples, upgrade tests, OpenAPI compatibility checks, and build verification.

